Exercise 1: Insert Documents and Query Data

Objective: Insert five documents with different structures into the orders collection and retrieve orders containing a specific product SKU.

First, we used insertMany() to insert five orders with different fields and nested structures. For example, some documents contained a coupon, shipping, payment, or tags field, while others did not.

We then used the following query to find all orders containing the product SKU BK-01:

```
None
db.orders.find({
   "items.sku": "BK-01"
})
```

Result: MongoDB returned five matching documents: orders 1001, 1003, 2001, 2003, and 2005.

Explanation: The items field is an array of embedded documents. MongoDB supports querying nested fields inside arrays using dot notation. This demonstrates the flexibility of the document model because documents in the same collection do not need to have exactly the same fields.

Exercise 2: Aggregation – Calculate Revenue by City

Objective: Calculate the total revenue for each city using the Aggregation Pipeline.

We used the $unwind stage to separate the items array into individual documents. Then, we used $group to group the orders by customer city.

The $multiply operator calculated the revenue of each item by multiplying its quantity by its price. The $sum operator calculated the total revenue for each city. Finally, $sort sorted the results in descending order of revenue.

Results:

City	Total Revenue
Hanoi	129

Da Nang 81.5

Ho Chi Minh 25

Total revenue: 235.5

Conclusion: Hanoi generated the highest revenue, followed by Da Nang and Ho Chi Minh. The Aggregation Pipeline allows us to process, group, calculate, and analyze data efficiently through multiple stages.

Exercise 3: Index and Query Performance

Objective: Compare query performance before and after creating an index on a collection containing 10,000 documents.

First, we inserted 10,000 documents into the performance_test collection. We queried the document with orderId = 5000 and used explain("executionStats") to analyze query performance.

Before creating an index:

- MongoDB used COLLSCAN (Collection Scan).
- totalDocsExamined was 10,000.
- totalKeysExamined was 0.
- nReturned was 1.
- Execution time was 4 ms.

Next, we created an ascending index on the orderId field:

```
None
db.performance_test.createIndex({ orderId: 1 })
```

We ran the same query again.

After creating an index:

- MongoDB used IXSCAN (Index Scan), followed by FETCH.
- totalDocsExamined decreased to 1.
- totalKeysExamined was 1.
- nReturned remained 1.

- Execution time was 2 ms.

Conclusion:

Creating an index significantly reduced the number of documents examined, from 10,000 to 1. This demonstrates that indexes improve query efficiency by helping MongoDB locate documents without scanning the entire collection.

However, indexes require additional storage and increase the cost of insert, update, and delete operations.

Screenshot 3 exercises:



```
  {
    _id: 2001,
    customer: { name: 'An', city: 'Hanoi' },
    status: 'paid',
    coupon: 'SALE10',
    items: [ { sku: 'BK-01', title: 'Python Basics', qty: 2, price: 12.5 } ]
  },
  {
    _id: 2002,
    customer: { name: 'Binh', city: 'Da Nang' },
    status: 'shipped',
    shipping: { method: 'express' },
    items: [ { sku: 'BK-03', title: 'Data Viz', qty: 1, price: 22 } ]
  },
  {
    _id: 2003,
    customer: { name: 'Chi', city: 'Hanoi' },
    status: 'pending',
    items: [
      { sku: 'BK-01', title: 'Python Basics', qty: 1, price: 12.5 },
      { sku: 'BK-07', title: 'SQL in Practice', qty: 2, price: 18 }
    ],
    tags: [ 'student', 'priority' ]
  },
  {
    _id: 2004,
    customer: { name: 'Dung', city: 'Ho Chi Minh' },
    status: 'paid',
    payment: { method: 'card', paid: true },
    items: [ { sku: 'BK-05', title: 'MongoDB Basics', qty: 1, price: 25 } ]
  },
  {
    _id: 2005,
    customer: { name: 'Em', city: 'Da Nang' },
    status: 'pending',
    note: 'Deliver after 5 PM',
    items: [ { sku: 'BK-01', title: 'Python Basics', qty: 3, price: 12.5 } ]
  }
]
shop> db.orders.find({
|   "items.sku": "BK-01"
| })
[
  {
    _id: 1001,
    customer: { name: 'Linh', city: 'Hanoi' },
    status: 'shipped',
    items: [
      { sku: 'BK-01', title: 'Python Basics', qty: 2, price: 12.5 },
      { sku: 'BK-07', title: 'SQL in Practice', qty: 1, price: 18 }
    ],
    tags: [ 'student', 'first-order' ]
  },
  {
    _id: 1003,
    customer: { name: 'Mai', city: 'Hanoi' },
    status: 'paid',
    items: [ { sku: 'BK-01', title: 'Python Basics', qty: 1, price: 12.5 } ],
    reviewed: false
  },
  {
    _id: 2001,
    customer: { name: 'An', city: 'Hanoi' },
    status: 'paid',
```

```
    _id: 2001,
    customer: { name: 'An', city: 'Hanoi' },
    status: 'paid',
    coupon: 'SALE10',
    items: [ { sku: 'BK-01', title: 'Python Basics', qty: 2, price: 12.5 } ]
  },
  {
    _id: 2003,
    customer: { name: 'Chi', city: 'Hanoi' },
    status: 'pending',
    items: [
      { sku: 'BK-01', title: 'Python Basics', qty: 1, price: 12.5 },
      { sku: 'BK-07', title: 'SQL in Practice', qty: 2, price: 18 }
    ],
    tags: [ 'student', 'priority' ]
  },
  {
    _id: 2005,
    customer: { name: 'Em', city: 'Da Nang' },
    status: 'pending',
    note: 'Deliver after 5 PM',
    items: [ { sku: 'BK-01', title: 'Python Basics', qty: 3, price: 12.5 } ]
  }
]
shop> db.orders.aggregate([
|   // Step 1: Separate each item
|   {
|     $unwind: "$items"
|   },
|
|   // Step 2: Group by city and calculate revenue
|   {
|     $group: {
|       _id: "$customer.city",
|       totalRevenue: {
|         $sum: {
|           $multiply: [
|             "$items.qty",
|             "$items.price"
|           ]
|         }
|       }
|     }
|   },
|
|   // Step 3: Sort by revenue descending
|   {
|     $sort: {
|       totalRevenue: -1
|     }
|   }
| ])
[
  { _id: 'Hanoi', totalRevenue: 129 },
  { _id: 'Da Nang', totalRevenue: 81.5 },
  { _id: 'Ho Chi Minh', totalRevenue: 25 }
]
shop> use shop
[
| db.performance_test.drop()
|
| let testData = [];
|
| for (let i = 1; i <= 10000; i++) {
```

```
| for (let i = 1; i <= 10000; i++) {
|   testData.push({
|     orderId: i,
|     customer: "Customer_" + i,
|     city: ["Hanoi", "Da Nang", "Ho Chi Minh"][i % 3],
|     status: i % 2 === 0 ? "paid" : "pending",
|     amount: i * 10
|   });
| }
|
| db.performance_test.insertMany(testData);
already on db shop
shop> db.performance_test.countDocuments()
0
shop> db.performance_test.insertMany(testData);
ReferenceError: testData is not defined
shop> show collections
orders
shop> let testData = [];
[
| for (let i = 1; i <= 10000; i++) {
|   testData.push({
|     orderId: i,
|     customer: "Customer_" + i,
|     city: ["Hanoi", "Da Nang", "Ho Chi Minh"][i % 3],
|     status: i % 2 === 0 ? "paid" : "pending",
|     amount: i * 10
|   });
| }
|
| db.performance_test.insertMany(testData);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae4cbd3e11229e93cacc5c0'),
    '1': ObjectId('6ae4cbd3e11229e93cacc5c1'),
    '2': ObjectId('6ae4cbd3e11229e93cacc5c2'),
    '3': ObjectId('6ae4cbd3e11229e93cacc5c3'),
    '4': ObjectId('6ae4cbd3e11229e93cacc5c4'),
    '5': ObjectId('6ae4cbd3e11229e93cacc5c5'),
    '6': ObjectId('6ae4cbd3e11229e93cacc5c6'),
    '7': ObjectId('6ae4cbd3e11229e93cacc5c7'),
    '8': ObjectId('6ae4cbd3e11229e93cacc5c8'),
    '9': ObjectId('6ae4cbd3e11229e93cacc5c9'),
    '10': ObjectId('6ae4cbd3e11229e93cacc5ca'),
    '11': ObjectId('6ae4cbd3e11229e93cacc5cb'),
    '12': ObjectId('6ae4cbd3e11229e93cacc5cc'),
    '13': ObjectId('6ae4cbd3e11229e93cacc5cd'),
    '14': ObjectId('6ae4cbd3e11229e93cacc5ce'),
    '15': ObjectId('6ae4cbd3e11229e93cacc5cf'),
    '16': ObjectId('6ae4cbd3e11229e93cacc5d0'),
    '17': ObjectId('6ae4cbd3e11229e93cacc5d1'),
    '18': ObjectId('6ae4cbd3e11229e93cacc5d2'),
    '19': ObjectId('6ae4cbd3e11229e93cacc5d3'),
    '20': ObjectId('6ae4cbd3e11229e93cacc5d4'),
    '21': ObjectId('6ae4cbd3e11229e93cacc5d5'),
    '22': ObjectId('6ae4cbd3e11229e93cacc5d6'),
    '23': ObjectId('6ae4cbd3e11229e93cacc5d7'),
    '24': ObjectId('6ae4cbd3e11229e93cacc5d8'),
    '25': ObjectId('6ae4cbd3e11229e93cacc5d9'),
    '26': ObjectId('6ae4cbd3e11229e93cacc5da'),
    '27': ObjectId('6ae4cbd3e11229e93cacc5db'),
    '28': ObjectId('6ae4cbd3e11229e93cacc5dc'),
    '29': ObjectId('6ae4cbd3e11229e93cacc5dd'),
```

```
    '9971': ObjectId('6ae4cbd3e11229e93cacecb3'),
    '9972': ObjectId('6ae4cbd3e11229e93cacecb4'),
    '9973': ObjectId('6ae4cbd3e11229e93cacecb5'),
    '9974': ObjectId('6ae4cbd3e11229e93cacecb6'),
    '9975': ObjectId('6ae4cbd3e11229e93cacecb7'),
    '9976': ObjectId('6ae4cbd3e11229e93cacecb8'),
    '9977': ObjectId('6ae4cbd3e11229e93cacecb9'),
    '9978': ObjectId('6ae4cbd3e11229e93cacecba'),
    '9979': ObjectId('6ae4cbd3e11229e93cacecbb'),
    '9980': ObjectId('6ae4cbd3e11229e93cacecbc'),
    '9981': ObjectId('6ae4cbd3e11229e93cacecbd'),
    '9982': ObjectId('6ae4cbd3e11229e93cacecbe'),
    '9983': ObjectId('6ae4cbd3e11229e93cacecbf'),
    '9984': ObjectId('6ae4cbd3e11229e93cacecc0'),
    '9985': ObjectId('6ae4cbd3e11229e93cacecc1'),
    '9986': ObjectId('6ae4cbd3e11229e93cacecc2'),
    '9987': ObjectId('6ae4cbd3e11229e93cacecc3'),
    '9988': ObjectId('6ae4cbd3e11229e93cacecc4'),
    '9989': ObjectId('6ae4cbd3e11229e93cacecc5'),
    '9990': ObjectId('6ae4cbd3e11229e93cacecc6'),
    '9991': ObjectId('6ae4cbd3e11229e93cacecc7'),
    '9992': ObjectId('6ae4cbd3e11229e93cacecc8'),
    '9993': ObjectId('6ae4cbd3e11229e93cacecc9'),
    '9994': ObjectId('6ae4cbd3e11229e93cacecca'),
    '9995': ObjectId('6ae4cbd3e11229e93caceccb'),
    '9996': ObjectId('6ae4cbd3e11229e93caceccc'),
    '9997': ObjectId('6ae4cbd3e11229e93caceccd'),
    '9998': ObjectId('6ae4cbd3e11229e93cacecce'),
    '9999': ObjectId('6ae4cbd3e11229e93caceccf')
  }
}
shop> db.performance_test.countDocuments()
10000
shop> db.performance_test.find({
|   orderId: 5000
| }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'shop.performance_test',
    parsedQuery: { orderId: { '$eq': 5000 } },
    indexFilterSet: false,
    queryHash: '0A982FEE',
    planCacheShapeHash: '0A982FEE',
    planCacheKey: '8B422CEE',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { orderId: { '$eq': 5000 } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 4,
    totalKeysExamined: 0,
    totalDocsExamined: 10000,
```

```
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 10000
    }
  },
  queryShapeHash: '1FD9B97F606F8184D63DD8BC6554D7EF20DF96C30726FFCA42C7D810E4A42E28',
  command: {
    find: 'performance_test',
    filter: { orderId: 5000 },
    '$db': 'shop'
  },
  serverInfo: {
    host: 'c62845b468fb',
    port: 27017,
    version: '8.0.34',
    gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
shop> db.performance_test.createIndex({ orderId: 1 })
orderId_1
shop> db.performance_test.find({
|   orderId: 5000
| }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'shop.performance_test',
    parsedQuery: { orderId: { '$eq': 5000 } },
    indexFilterSet: false,
    queryHash: '0A982FEE',
    planCacheShapeHash: '0A982FEE',
    planCacheKey: '38622587',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { orderId: 1 },
        indexName: 'orderId_1',
        isMultiKey: false,
        multiKeyPaths: { orderId: [] },
        isUnique: false,
        isSparse: false,
```

```
        stage: 'IXSCAN',
        keyPattern: { orderId: 1 },
        indexName: 'orderId_1',
        isMultiKey: false,
        multiKeyPaths: { orderId: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { orderId: [ '[5000, 5000]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 2,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 1,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 1,
        executionTimeMillisEstimate: 0,
        works: 2,
        advanced: 1,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { orderId: 1 },
        indexName: 'orderId_1',
        isMultiKey: false,
        multiKeyPaths: { orderId: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { orderId: [ '[5000, 5000]' ] },
        keysExamined: 1,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: '1FD9B97F606F8184D63DD8BC6554D7EF20DF96C30726FFCA42C7D810E4A42E28',
```

```
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 1,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 1,
        executionTimeMillisEstimate: 0,
        works: 2,
        advanced: 1,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { orderId: 1 },
        indexName: 'orderId_1',
        isMultiKey: false,
        multiKeyPaths: { orderId: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { orderId: [ '[5000, 5000]' ] },
        keysExamined: 1,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: '1FD9B97F606F8184D63DD8BC6554D7EF20DF96C30726FFCA42C7D810E4A42E28',
  command: {
    find: 'performance_test',
    filter: { orderId: 5000 },
    '$db': 'shop'
  },
  serverInfo: {
    host: 'c62845b468fb',
    port: 27017,
    version: '8.0.34',
    gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```